Improving Students' Code Comprehension Ability Through LLM-Generated Practice Problems

William Jordan Kwako
wkwako3@gatech.edu

***Abstract*—**This paper discusses the design, evaluation, and ongoing development of a web-based app to improve students' ability to read code through solving practice problems. These problems are generated via an LLM, and focus on testing users' code comprehension skills. Questions include asking students to determine the output of a block of code, and renaming generic functions to improve documentation. The query for the LLM contains requirements and restrictions to guarantee a consistent structure in generated code. A survey was administered to participants following a brief use of the website. 82% of participants said that they would enjoy using this website as part of coursework, while 70% of participants agreed or strongly agreed that this website could help improve their ability to read code.

1 INTRODUCTION

The stronger a student's grasp on core computer science (CS) concepts, the higher their overall adaptability and performance in a professional environment. While employers often find that new software developers' technical skills are sufficient for work (Begel and Simon, 2008), this is not necessarily indicative of learning objectives being met in class. For example, students may be refining their skills and filling in gaps in their education post-graduation. Starting a job at any medium to large software company entails familiarizing oneself with the code base, which could be thousands to hundreds of thousands of lines long. It is crucial that students feel confident reading code that others have written, particularly when refactoring, a common practice in the CS industry. The demand for competent computer scientists will continue to grow. Maximizing the efficiency and effectiveness with which computer scientists are trained will increase the quality of current and future technological endeavors.

The created web app seeks to solve the issue of students failing to meet learning objectives by giving students practice reading code through solving LLM-generated problems. Problem types – how the user solves the problem – include

reading a block of code and determining its output, and editing function names to clarify their purpose. Problem topics – the domain from which the code originates – include STEM subjects like CS and math. Users are more likely to receive CS problems, but may occasionally see physics or geoscience ones. Practice is crucial to developing CS skills, and is especially important in introductory-level classes. My tool provides a new method of practice, and particularly focuses on developing a learner's ability to read code.

2 RELATED WORK

2.1 Learning to Read Code

Researchers have identified that students particularly struggle to read code, which may partially explain the discrepancy between graduating students' skills and CS learning objectives. As cited in (Simon et al., 2009), Denny et al. (2008) discovered that students tend to perform better on writing problems than code tracing problems, that is, where students read through a block of code line-by-line, analyzing which variables change and where. Reading code is often not directly taught in CS classes, even though the ability to read code precedes the ability to write it (Lister et al., 2009). Additionally, some instructors are reluctant to introduce code-tracing activities into the classroom, as indicated by instructor survey responses (Busjahn and Schulte, 2013). Notably, there is a strong connection between learning to read and write code, as Lister et al. (2009) found that these skills generally grow in tandem. Other international studies corroborate these findings, such as Simon et al. (2009), who found that students' code reading and writing skills are roughly the same, but that top performers have stronger reading than writing skills, whereas others' reading skills lag behind their writing skills. Furthermore, eye-tracking studies suggest that novice CS learners may read code differently than experts do. Students may have difficulty reading code in the same way as CS professionals, potentially reflecting a lack of instruction. However, there is some conflicting evidence on this front, as other eye-tracking studies indicate no differences between how novices and experts read code (Peitek et al., 2020).

Quantitative assessments portray a similar view of students' lack of code comprehension skills. In one study, participants scored an average of 21% on a post-introductory CS course exam designed to assess their ability to code (McCracken et al., 2001). In another, students who had just taken an introductory CS class

were asked to read a snippet of code, then write code that performed the opposite action (Teague and Lister, 2014). For example, if the provided code shifted all elements in a list to the right, they were asked to write code that shifted all elements in a list to the left. The vast majority of students were unable to perform this task, with success rates varying from 8% to 32% by cohort.

2.2 Educational Apps for Developing CS Skills

There is a lack of publicly available tools for students to practice code reading skills. Websites like LeetCode, Project Euler, and HackerRank offer methods to hone students' CS skills through coding challenges. Others incorporate Generative AI into their platform, like Denigma, which analyzes a block of code and explains its purpose in plain language. However, these sites only check for solution validity, and teach students how to solve new problems instead of improving students' code reading skills.

Related tools, such as Code Mangler, uses Generative AI to create exam questions by breaking a block of code – rearranging lines and adding bugs (Cheng and Harrington, 2017). These problems tend to focus on reading code instead of writing it, as students are required to find and fix the bugs in the code. Students who scored well on these questions also scored well on conventional exam questions, demonstrating Code Mangler's validity.

3 PROJECT DESIGN

3.1 Generative AI Overview

Generative AI (GenAI) is a specialized kind of AI that can create new text, images, audio, and other forms of media. These GenAI models are trained on existing data, with original source examples that include books, articles, research papers, music, and websites. Many other forms of training data can be utilized depending on the intended use of the model and the target audience. Early forms of AI have existed since the 1960s, but a slew of recent technological breakthroughs has caused the ubiquity seen today (Foote, 2024). Large language models like ChatGPT and Gemini – robust GenAI models – draw on large amounts of data to recognize and respond to text written by humans. Since this increase in popularity, more specialized models have emerged and been applied across a wide variety of fields. Education is one of the primary domains in which GenAI could see the greatest benefit if employed appropriately. These tools could assist CS

instructors by generating homework problems and exam questions, and being a source of information from which students can learn. Ultimately, GenAI may help students meet CS learning objectives in introductory-level courses.

3.2 A Tool Designed to Develop Code Comprehension Skills

Rather than asking students to write code themselves, my project seeks to create problems that rely on a student's code comprehension ability to solve. This instructional design choice is supported by evidence from Lister et al. (2009), who recommend that instructors incorporate activities involving reading and tracing code into computer science curricula. ChatGPT is used to generate problems of varying types, topics, and difficulty levels. A hint and explanation system is included to guide students toward correct answers.

Two different types of problems are randomly generated, with equal probability, for users to solve. In the first, users must determine what output a block of code produces. A text box is provided for users to input their answer. In the second, functions are given generic names with incrementing numerical values, like 'mystery1' and 'mystery2,' and users must edit the code to rename them more appropriately. Once submitted, the block of code is sent to ChatGPT, which is asked to verify the suitability of function names. There are many correct answers, and ChatGPT is asked to be lenient with grading. For both problem types, code is validated on the backend for syntax errors, logical errors, and malicious code before being shown to users.

The web app tailors to students of varying expertise levels. Three difficulty levels are available to users: easy, medium, and hard. These levels are primarily distinguished by length of code and use of Pythonic syntax, like list and dictionary comprehensions. They are only general guidelines however, and there is intended to be some overlap. "Easy" problems are designed for introductory CS students learning programming for the first time. "Medium" problems are designed for students that have a couple of years of experience, and "hard" problems are designed for those proficient in CS, approximately at the skill level of entry-level software engineers. In Figure 1, an image of the website's practice page is displayed, along with a generated 'medium' difficulty problem. If a user finds a problem too challenging, a hint system is available that helps them progress towards the solution. Hints are generated dynamically based on user input, being generic if the user has not attempted the problem, but specific to

the recorded input if the user has attempted the problem. Additionally, if a user solves the problem correctly without understanding the solution, an explanation button can optionally be used to provide rationale for the answer.

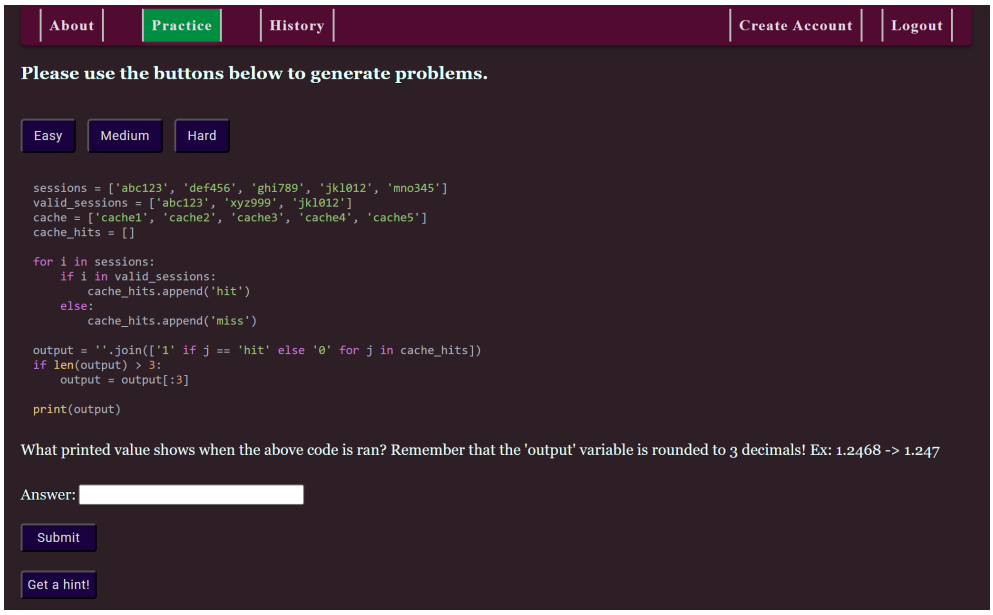

***Figure 1*—A picture of the 'practice' page on the website. A 'medium' difficulty problem has been generated, which requires the user to determine the output to solve.**

A history page is available to users that displays their 20 most recently attempted problems, which can be seen in Figure 2. Data such as problem type, difficulty level, and result are stored, helping users track their progress over time. Only the first attempt is recorded, so if a user initially gets the wrong answer but subsequently gets it correct, only the incorrect attempt will be displayed. Additionally, the number of correct answers is tabulated and displayed in the upper-right corner of this page, updating dynamically as the user solves problems.

3.3 Query Specifications and Problem Diversity

The query provided to ChatGPT ensures problems are generated with a consistent structure. Examples of requirements and constraints include: variable and function names must be human readable, `i`, `j`, and `k` must be used as iterators in loops, the number of iterations through an inner `for` loop must be 5 or fewer, the code must not have any docstrings or annotations, and `input()` and `random()` functions are disallowed.

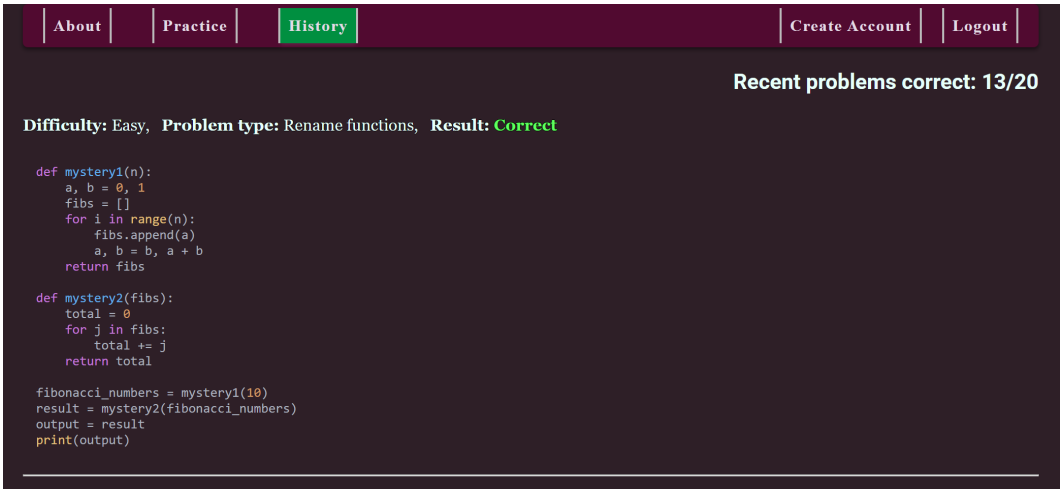

Figure 2—A picture of the 'history' page on the website. The user's 20 most recently attempted problems are displayed. Each problem's difficulty, problem type, and result are stored. The user's progress is tracked in the upper-right corner.

Despite the requirements and constraints provided in the query, ChatGPT still often produces problems with similar content, indicating a need for methods to enhance problem variety. A list of domains, subjects, concepts, and question types is compiled, then one is randomly picked from each list and included in the query. Domains are broad fields such as math, computer science, and physics; subjects are subfields within these categories, such as mechanics, cryptography, and bitwise operations; concepts refer to what the code is doing, such as validating systems or classifying inputs; and finally, question types are actions the user takes to investigate the code, such as pattern detection or tracking a running total. After each attribute has been selected, they are combined into a sentence and added to the query. Plain language is combined with variables that represent each attribute using 'f-strings', a Pythonic way to join string literals and variables: `query += f"Use an example from {domain} related to {chosen_subject} involving {chosen_concept} and {chosen_q_type}"`. There are hundreds of thousands of strings that can be formed by combining attributes in this way, ensuring a large number of unique problems for users. Additionally, ChatGPT's output can vary depending on small changes in query phrasing, so adjectives from a list are randomly incorporated into the query to help diversify the output.

3.4 Software Specifications

The web app is built with HTML, CSS, JavaScript, and Python, and is hosted on Pythonanywhere, a platform that allows users to deploy web-based Python environments. Django serves as the fullstack framework, enabling communication between Python and the frontend components. Several modules are included in Django by default that streamline development of common web-related features, like authentication and concurrent request handling. The web app is publicly available at: https://wkwako.pythonanywhere.com/.

4 PROJECT EVALUATION

4.1 Survey Sample and Administration

A survey was created that gathered feedback on the website from computer scientists. Participants are either students at OMSCS or work in CS careers. Each participant was asked to spend 10-15 minutes using the website, noting both helpful features and areas of improvement. Participants were asked about the layout, problem diversity, the feedback system, and whether or not they would use this website outside of course requirements. 17 participants responded to the survey. Below, results for the Likert-scale questions are shown. Responses to short-answer questions are discussed in more detail in the Survey Results section.

Table 1—Left: 88% of participants agreed or strongly agreed the code written by ChatGPT was human like. Right: 70% of participants agreed or strongly agreed that this website could help improve their ability to read code. No participants disagreed or strongly disagreed with this statement.

The code generated by ChatGPT was human like	**# responses**
Strongly disagree	0
Disagree	0
Neutral	2
Agree	12
Strongly agree	3

The website could help me improve my ability to read code	**# responses**
Strongly disagree	0
Disagree	0
Neutral	5
Agree	9
Strongly agree	3

Table 2—Left: 82% of participants said they would enjoy using this website as part of a CS course. Right: 53% of participants said they would use this website for personal CS skill development. Several participants explicitly said that they do not seek professional development outside coursework, so this may explain the discrepancy between these two survey questions.

I would enjoy using this website as part of a CS course	# responses
Strongly disagree	0
Disagree	3
Neutral	3
Agree	8
Strongly agree	3

I would use this website outside of course requirements	# responses
Strongly disagree	1
Disagree	2
Neutral	5
Agree	5
Strongly agree	4

Table 3—Left: 71% of participants said that the difficulty of the questions lined up with the respective options ("Easy", "Medium", or "Hard"). One partipant strongly disagreed, suggesting they had a negative experience significantly worse than the other participants. Right: 59% of participants thought that there was enough problem variety.

The difficulty lined up with the respective options	# responses
Strongly disagree	1
Disagree	0
Neutral	4
Agree	9
Strongly agree	3

There was enough problem variety	# responses
Strongly disagree	0
Disagree	2
Neutral	5
Agree	8
Strongly agree	2

4.2 Survey Results

As seen in Table 1, 88% of participants agreed that the code written by ChatGPT was human like. In the relevant short-answer section, several students emphasized these thoughts. More specific comments included: "It was elegant code. It took some time for me to interpret, though, as I am not as familiar with python", and "perhaps not the most optimized code ever, but code that humans would write." This indicates that ChatGPT's code successfully mimics that of a human's.

Table 4—Left: 62% of participants said that the hints were helpful. Note that participants that did not utilize the hints feature were not required to respond to this question, so the number of total responses is lower than the number of responses to other questions. Right: 88% of participants said that the website was easy to navigate.

The hints were helpful	# responses
Strongly disagree	1
Disagree	1
Neutral	3
Agree	5
Strongly agree	3

The website was straightforward to navigate	# responses
Strongly disagree	0
Disagree	0
Neutral	2
Agree	8
Strongly agree	7

As seen in Table 2, 82% of participants said they would enjoy using this website as part of coursework in CS. However, several participants expressed concerns about learning from an LLM. One student said, "I think [a] professor needs to design and select the questions, not [let] a tool randomly generate them." Another participant expressed dissatisfaction about LLMs grading students. But conversely, many participants thought this was a great tool that professors should encourage for practice. 53% of participants said they would use this website for personal CS use. Several participants explicitly said that they do not seek professional skill development outside of coursework, which may explain the discrepancy in survey responses between using this tool for coursework versus personal use. In Table 1, participants rate the overall utility of the website. 70% of participants agreed that this website could help improve their ability to read code. The majority of students found the exercises helpful, indicating that the initial version of the website was a success.

Tables 3 and 4 display results from survey questions regarding difficulty, problem variety, hint usage, and website layout. 70% of participants agreed or strongly agreed that problem difficulty matched difficulty level of the available buttons ("Easy", "Medium", "Hard"). One participant strongly disagreed that the difficulty options lined up with the generated code, indicating that they had an exceptionally negative experience compared to the other participants. Problems are randomly generated, so it is possible that this student received a difficult

problem expecting an easier one, or ran into bugs that prevented them from finding a solution. 59% of participants thought problem variety was sufficient, though participants likely did not use the website for long enough to see duplicate questions. Additionally, many students were dissatisfied with the number of math-related questions and suggested incorporating more CS-related questions instead, particularly those relevant to the tech sector.

62% of participants described the hints as helpful, and several provided feedback with short-answer responses specifically about the hints system. One responder said that the hints were incorrect, while another mentioned a desire to see an explanation for the correct answer after getting the problem right, and potentially after several incorrect attempts as well. Similarly, 88% of participants thought that the website was easy to navigate. This was the highest ratio of agree to disagree responses, indicating that the website layout was one of its most well-received attributes. However, some comments mentioned that the color combination of the dark gray background and the dark purple text did not contrast enough, decreasing legibility.

5 LIMITATIONS

5.1 Narrow Scope of Problem Diversity, Topics, and Languages

The initial reception of the website was positive, and several features desired by participants, such as color scheme modifications and answer rationale, have already been incorporated. However, there is still room for improvement, and several features could be more robust to provide a more comprehensive experience for the user. First, the core functionality of the website can be enhanced. As mentioned previously, there are two types of problems: those in which the user determines the output of the code, and those in which the user must rename functions. Topics for these types of problems are determined by randomizing lists of domains, subjects, concepts, and problem types. Expanding the amount of problem types and refining the list of topics may broaden a user's CS skill set, and contribute to better learning outcomes.

The website presently only applies to Python learners, when it should instead be as broadly applicable as possible. Python is a popular language, but adding other commonly used languages like Java, C, C#, and Javascript, would help the website reach a wider audience. Similarly, customization options could help

personalize each user's experience. Users may be more willing to try the website if they are allowed more control, like disallowing syntactical features like list and dictionary comprehensions, or adding their own domain of study.

5.2 Motivation Potential With Gamification

As indicated in the survey responses, many students do not study CS independent of coursework. While this could be due to a number of underlying factors such as interest, motivation, or lack of skill or time, gamification could enhance users' experience and provide an incentive for individual study, increasing the number of students that study for personal or professional gain. Gamification is the addition of 'game-like' elements to an activity, like points, competitive leaderboards, detailed statistics for tracking user progress, and achievements. There are numerous studies indicating that incorporating it correctly into academic environments increases students' motivation and learning outcomes. For example, Papastergiou (2009) investigates the effects of a computer game's effectiveness for teaching students about various pieces of computer hardware, revealing an increase in motivation and rate of learning, regardless of gender. And specifically within CS, leaderboard tools have been found to spark interest, and cause students to finish homework assignments more quickly (Call et al., 2021). Similar elements could be added to my web app to bolster student engagement.

5.3 Lack of Quantitative Experimentation

Despite positive survey results, my web app lacks quantitative data demonstrating its success. Nevertheless, my app has a unique opportunity to contribute to CS education, given the absence of studies comparing the benefits of reading code as opposed to writing it. Quantitative data would solidify this website as a worthwhile tool to CS learners. Introducing this tool to an undergraduate class and comparing a control group's scores to an experimental group's scores would be a practical method for testing its efficacy in a real-world environment. However, withholding a beneficial educational tool from students in the control group may be unethical, so alternative approaches may be necessary.

6 CONCLUSION

Tools that improve students' ability to read and understand code are scarce in CS education, even though there are many studies demonstrating the effectiveness of reading and tracing code. The web app aims to help fill this educational gap

by generating problems via ChatGPT for students to solve. The difficulty options accommodate students from a wide variety of expertise levels, and combining attributes like domains, subjects, concepts, and question types ensures a diverse problem set. Hints and post-exercise explanations are available to students on request, which further cement their understanding of these coding problems.

Initial feedback is positive, with many survey participants noticing the intended application and commenting on its usefulness. Constructive feedback is equally valuable, and provides actionable next steps to improve functionality. The color scheme, math-heavy problems, and variability in difficulty are features that were not well received. Areas of improvement include advanced statistic tracking, gamification features, and additional language support. Furthermore, quantitative data from formal experimentation in a classroom setting would further demonstrate the effectiveness of the app. With positive results, it would be another addition to a growing body of work demonstrating that reading code is an effective method for learning computer science.

7 REFERENCES

[1] Begel, A. and Simon, B. (Mar. 2008). "Struggles of new college graduates in their first software development job". In: *Proceedings of the 39th SIGCSE technical symposium on Computer science education*. SIGCSE '08. ACM. DOI: 10.1145/1352135.1352218. URL: http://dx.doi.org/10.1145/1352135.1352218.

[2] Busjahn, T. and Schulte, C. (Nov. 2013). "The use of code reading in teaching programming". In: *Proceedings of the 13th Koli Calling International Conference on Computing Education Research*. Koli Calling '13. ACM, pp. 3–11. DOI: 10.1145/2526968.2526969. URL: http://dx.doi.org/10.1145/2526968.2526969.

[3] Call, Madison W, Fox, Erik, and Sprint, Gina (Nov. 2021). "Gamifying software engineering tools to motivate computer science students to start and finish programming assignments earlier". In: *IEEE Trans. Educ.* 64.4, pp. 423–431.

[4] Cheng, N. and Harrington, B. (Mar. 2017). "The Code Mangler: Evaluating Coding Ability Without Writing any Code". In: *Proceedings of the 2017 ACM SIGCSE Technical Symposium on Computer Science Education*. SIGCSE '17.

ACM, pp. 123–128. DOI: 10.1145/3017680.3017704. URL: http://dx.doi.org/10.1145/3017680.3017704.

[5] Denny, P., Luxton-Reilly, A., and Simon, B. (Sept. 2008). "Evaluating a new exam question: Parsons problems". In: *Proceedings of the Fourth international Workshop on Computing Education Research*. ICER '08. ACM. DOI: 10.1145/1404520.1404532. URL: http://dx.doi.org/10.1145/1404520.1404532.

[6] Foote, Keith D. (2024). *A Brief History of Generative AI - DATAVERSITY — dataversity.net*. https://www.dataversity.net/a-brief-history-of-generative-ai/. [Accessed 02-02-2025].

[7] Lister, R., Fidge, C., and Teague, D. (July 2009). "Further evidence of a relationship between explaining, tracing and writing skills in introductory programming". In: *ACM SIGCSE Bulletin* 41.3, pp. 161–165. ISSN: 0097-8418. DOI: 10.1145/1595496.1562930. URL: http://dx.doi.org/10.1145/1595496.1562930.

[8] McCracken, M., Almstrum, V., Diaz, D., Guzdial, M., Hagan, D., Kolikant, Y. B.D., Laxer, C., Thomas, L., Utting, I., and Wilusz, T. (Dec. 2001). "A multi-national, multi-institutional study of assessment of programming skills of first-year CS students". In: *ACM SIGCSE Bulletin* 33.4, pp. 125–180. ISSN: 0097-8418. DOI: 10.1145/572139.572181. URL: http://dx.doi.org/10.1145/572139.572181.

[9] Papastergiou, Marina (Jan. 2009). "Digital Game-Based Learning in high school Computer Science education: Impact on educational effectiveness and student motivation". en. In: *Comput. Educ.* 52.1, pp. 1–12.

[10] Peitek, Norman, Siegmund, Janet, and Apel, Sven (July 2020). "What drives the reading order of programmers?" In: *Proceedings of the 28th International Conference on Program Comprehension*. Seoul Republic of Korea: ACM.

[11] Simon, Lopez, M., Sutton, K., and Clear, T. (2009). "Surely We Must Learn to Read before We Learn to Write!" In: *Proceedings of the Eleventh Australasian Computing Education Conference*. ACE 2009. ACS.

[12] Teague, D. and Lister, R. (2014). "Programming: reading, writing and reversing". In: *Proceedings of the 2014 conference on Innovation and technology in computer science education - ITiCSE '14*. ITiCSE '14. ACM Press, pp. 285–290. DOI: 10.1145/2591708.2591712. URL: http://dx.doi.org/10.1145/2591708.2591712.